

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch20_pretrained_lms

AYuSh

Contents

Chapter 20: Pretrained Language Models	3
--	---

Chapter 20: Pretrained Language Models

The single most consequential idea in modern NLP fits in one sentence: *train a big model on unlabeled text with a self-supervised objective, then adapt it cheaply to every downstream task*. This chapter demonstrates that idea rather than describing it — an entire pretrain-then-transfer pipeline built from scratch and *measured*, using the shared miniature transformer of Listing 1 (batched forward/backward, gradient-checked to 5×10^{-10}). The point of the miniatures is calibration: some famous properties reproduce at 330k parameters, others visibly require scale, and knowing which is which is exactly the judgment interviews test.

The measurements: an MLM-pretrained encoder whose masked-word predictions expose what small models learn first — the corpus prior, with topical knowledge visible only after frequency adjustment (Listing 2); the headline transfer result — with 50 labels, the pretrained encoder scores **0.781 against 0.563 from scratch**, the same architecture separated only by 4,000 steps of filling in masked words on unlabeled text (Listing 3); a GPT-style causal LM beating Chapter 19's best classical model, **perplexity 134 vs 185** on identical data and tokenization (Listing 4); an induction head grown in a 2-layer model — **99.7% copy accuracy on sequences it cannot have memorized** (40^8 possibilities), with the attention stripe photographed (Listing 5); a frozen-feature probe where causal features *beat* bidirectional ones at equal step count — the honest sample-efficiency counterpoint to the BERT narrative (Listing 6); SBERT's lesson in miniature — contrastive fine-tuning lifting same-document retrieval from $8\times$ to $11\times$ chance in 300 steps (Listing 7); and distillation dissected — the student gains almost nothing from soft labels on the same 200 examples but jumps $0.632 \rightarrow 0.807$ when the teacher labels 3,000 *unlabeled* chunks (Listing 8).

BERT: masked-language-model pretraining

BERT (Devlin et al., 2018) is Chapter 16's *encoder* — bidirectional attention, no causal mask — trained on a cloze task: mask 15% of tokens, predict them from both sides. The masking recipe interviews probe (Listing 2 implements it exactly): of selected positions, 80% become `<mask>`, 10% a random token, 10% stay unchanged — because `<mask>` never appears at fine-tuning time, the model must keep real-token representations predictive rather than learning "only masked positions matter"; the random 10% keeps it honest about unchanged tokens. The second pretraining task, **next-sentence prediction**, is a cautionary tale: RoBERTa showed it *hurt* — dropping NSP, training longer on more data with bigger batches and dynamic masking beat BERT with zero architectural change, the cleanest demonstration that training recipes matter as much as architectures. The family

tree in one pass: **RoBERTa** (recipe fixes), **DistilBERT** (Listing 8's subject: 40% smaller via distillation, 97% of the quality), **DeBERTa** (disentangled content/position attention plus relative positions — the strongest of the line), **ALBERT** (parameter sharing across layers, embedding factorization).

Listing 2's miniature earns a calibration lesson the papers don't advertise: at 330k parameters and 4,000 steps, the raw cloze predictions are *function words* — the model learns the corpus prior first, and topical signal (*flames, nhl, round* for a hockey blank) appears only after subtracting log-frequency. BERT's famous cloze party trick is a property of scale, not of the objective.

The transfer experiment

Listing 3 is the chapter's centerpiece because it is the *paradigm* measured under laboratory control: same architecture, same optimizer, same label budget — the only difference is initialization, random vs the MLM checkpoint. Results: 50 labels — 0.563 scratch vs **0.781** pretrained; 200 — 0.639 vs 0.793; 1,000 — 0.740 vs 0.829. Read the table sideways: the pretrained model with 50 labels beats the scratch model with 1,000 — pretraining is worth roughly a $20\times$ *label multiplier* here, and that multiplier is the business case for the entire pretrained-model industry. Why it works: MLM forces the encoder to build features that predict words from context — topic, syntax, local semantics — and those features are 90% of what a downstream classifier needs; fine-tuning just rotates the head into place. Fine-tuning practicalities worth reciting: small learning rates (the miniature uses $5e-4$; BERT-scale uses $2e-5$) because large steps destroy the pretrained features (catastrophic forgetting); a fresh head trained from zero; and the modern alternative of freezing everything (Listing 6's probing) or adapting low-rank slices (LoRA, Chapter 21).

GPT: autoregressive pretraining

The GPT line is Chapter 16's *decoder* — causal mask, next-token prediction on every position. Listing 4 trains one and settles a cross-chapter score: on the *same* sequences and vocabulary, the 2-block transformer reaches perplexity **134.3** against **184.7** for Chapter 19's best classical model (interpolated trigram) — the neural LM's win coming from representation sharing (similar words share embeddings; trigrams treat *shuttle* and *rocket* as strangers) and 16 tokens of context against 2. The generations are locally fluent, globally drifty — a fair miniature of GPT-1-era output. The architectural claim to make in interviews: CLM's supervision is *dense* (every token is a target, vs MLM's 15%) and its interface is universal — any task expressible as text continuation is in scope, which is why scaling this objective (GPT-2's zero-shot behaviors, GPT-3's few-shot) produced the in-context learning era. **T5 and BART** complete the taxonomy: encoder-decoder models with denoising objectives (T5: span corruption, everything cast as text-to-text; BART: arbitrary noising, strongest at summarization-style generation-with-

grounding) — bidirectional understanding of the input plus autoregressive generation of the output, at the price of two stacks.

In-context learning and induction heads

GPT-3's headline — describe a task in the prompt, get performance without weight updates — looks like magic until you ask *what circuit implements it*. The leading mechanistic answer (Anthropic's induction-heads work) is a two-layer attention pattern: a **previous-token head** writes "what preceded me" into each position, and an **induction head** then attends from the current token to *the position after its previous occurrence* and copies — `[A][B] ... [A] → [B]`. Listing 5 grows one: a 2-layer model trained on repeat-the-first-half sequences hits **99.7% on novel sequences** ($40^8 \approx 6.5 \times 10^{12}$ possibilities — memorization is arithmetically impossible) while remaining at chance on the unpredictable first half, and the layer-2 attention concentrates $4\times$ uniform mass exactly on the prev-occurrence+1 stripe (the figure). Honest caveat, worth volunteering: the miniature's fixed period means a positional shortcut could coexist with content-based lookup; the diagnostic stripe and novel-token generalization together are the evidence. The claim the literature makes at scale: induction heads emerge in a sharp phase change early in training, coincide with the onset of in-context learning, and generalize from literal copying to fuzzy pattern completion — the mechanistic seed of few-shot prompting (Chapter 23).

Encoder vs decoder: when to use which

The taxonomy question every LLM interview asks, now answerable with this chapter's own data. **Encoders (BERT-family)** for *understanding with a fixed output*: classification, NER, retrieval embeddings — bidirectional context is strictly more information per token, and there's no generation to do. **Decoders (GPT-family)** for *generation and everything-as-text*: dense supervision, universal interface, in-context learning at scale; the modern default even for classification (via instruction prompting) once models are large. **Encoder-decoders (T5/BART)** for input-output tasks with long inputs and structured outputs — translation, summarization — where cross-attention gives the generator unrestricted bidirectional access to the source. Listing 6 supplies the measured nuance: probing *frozen* features at equal pretraining steps, the causal model's mean-pooled features (0.769) actually edge the bidirectional model's (0.746), both far above the random-weight control (0.571) — because CLM took loss on 100% of tokens while MLM took loss on 15%, and at miniature scale sample efficiency dominates. MLM's bidirectional advantage is real but shows up at scale and under fine-tuning — a distinction most candidates state backwards or not at all. Also measured: for causal models, mean-pooling beats last-token features (0.769 vs 0.715) — the last token summarizes for *prediction*, not for the document.

Sentence embeddings

Token vectors don't average into good sentence vectors for free: Listing 7's mean-pooled MLM features retrieve same-document chunks at 0.085 — $8\times$ chance (0.011) but far from useful. **SBERT**'s insight: the *training signal*, not the encoder, is what's missing — fine-tune with a siamese objective that pulls related texts together. The listing implements the modern version (in-batch contrastive InfoNCE with temperature 0.07, gradients hand-derived through the L2-normalization tangent projection — Chapter 17's CLIP machinery, text-to-text), using "two chunks of the same document" as free positive pairs: 300 steps lift retrieval to 0.123 ($11\times$ chance, +45% relative), with *document-held-out* evaluation so no test document contributed pairs. Scale this recipe — better positives (paraphrases, QA pairs), hard negatives, big batches — and you get the sentence-transformer / E5 / GTE embedding models that power Chapter 24's retrieval. Cross-encoder vs bi-encoder completes the interview answer: bi-encoders (this listing) embed independently — $O(1)$ per query against a precomputed index; cross-encoders read the pair jointly — far more accurate, $O(\text{candidates})$ per query — hence retrieve-then-rerank.

Distillation

DistilBERT's recipe — student mimics teacher's soft logits (temperature-scaled KL) — is usually explained as "dark knowledge in the soft targets." Listing 8 decomposes the win with a controlled comparison. Teacher: the pretrained encoder fine-tuned on 1,000 labels (0.845). Student: $4\times$ thinner, half the depth. On the *same 200 labeled examples*, hard labels give 0.632 and soft labels 0.625 — dark knowledge alone bought nothing here. Let the teacher label 3,000 *unlabeled* chunks and the student hits **0.807** — 96% of the teacher at a fraction of the size. The real engine is the **transfer set**: a teacher converts unlabeled text into unlimited supervision, and soft targets are the medium (per-example uncertainty, 2 bits of hard label becoming a full distribution). That's also the honest reading of DistilBERT itself, which distilled on the whole pretraining corpus, and of the current LLM practice of training small models on frontier-model outputs — same mechanism, new vocabulary ("synthetic data"). Practical notes: match student capacity to the transfer set, $T \approx 2\text{-}4$, mix a hard-label term when gold labels exist, and remember the T^2 gradient factor (in the listing's KD gradient).

Code implementations

Listing 1 — common.py — the shared miniature transformer (batched, full backward, grad-checked 5e-10)

```
"""shared miniature-transformer machinery for chapter 20 (imported by the listings)"""
import re, numpy as np
from sklearn.datasets import fetch_20newsgroups

def load_corpus(vmax=2000, seqlen=32):
```

```

data = fetch_20newsgroups(subset='train',
categories=['sci.space', 'rec.sport.hockey'],
                        remove=('headers', 'footers', 'quotes'))

from collections import Counter
pairs = [(re.findall(r"[a-z]++", d.lower()), t) for d, t in zip(data.data,
data.target)]
pairs = [(d, t) for d, t in pairs if len(d) >= 8]
docs = [d for d, _ in pairs]
counts = Counter(w for d in docs for w in d)
words = [w for w, _ in counts.most_common(vmax-4)]
vocab = {w: i+4 for i, w in enumerate(words)}
vocab["<pad>"], vocab["<unk>"], vocab["<mask>"], vocab["<cls>"] = 0, 1, 2, 3
enc = lambda d: [vocab.get(w, 1) for w in d]
seqs, labels = [], []
for d, yy in pairs:
    ids = enc(d)
    for s in range(0, max(1, len(ids)-seqlen+1), seqlen):
        chunk = ids[s:s+seqlen]
        if len(chunk) >= 8:
            seqs.append(np.array(chunk + [0]*(seqlen-len(chunk))))
            labels.append(yy)
return np.array(seqs), np.array(labels), vocab

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e/e.sum(-1, keepdims=True)
def ln_f(x):
    mu = x.mean(-1, keepdims=True); xc = x-mu
    inv = 1/np.sqrt((xc**2).mean(-1, keepdims=True)+1e-6); return xc*inv, (xc, inv)
def ln_b(dy, c):
    xc, inv = c
    return inv*(dy - dy.mean(-1, keepdims=True) - xc*inv**2*(dy*xc).mean(-1,
keepdims=True))

class Tiny:
    """2-block transformer, batched (B, n) int inputs; causal flag picks GPT vs BERT
    attention."""
    def __init__(self, V, n, d=48, H=4, depth=2, causal=False, seed=0):
        r = np.random.default_rng(seed)
        self.V, self.n, self.d, self.H, self.depth, self.causal = V, n, d, H, depth,
causal
        self.dh = d//H
        P = dict(E=r.normal(0, .5, (V,d)), pos=r.normal(0, .1, (n,d)), Wu=r.normal(0, d**-.
5, (d,V)))
        for i in range(depth):
            for w, shp, sc in [('Wq', (d,d), d**-.5), ('Wk', (d,d), d**-.5), ('Wv',
(d,d), d**-.5),
                                ('Wo', (d,d), d**-.5), ('W1', (d,4*d), d**-.5), ('W2', (4*d,d),
(4*d)**-.5)]:
                P[f'{w}{i}'] = r.normal(0, sc, shp)
            self.P = P
            self.mask = np.where(np.tril(np.ones((n,n)))==1, 0.0, -1e9) if causal else 0.0
            self.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in P.items()}
            self.t = 0
        def fwd(self, x, pad_attn=True):
            P, H, dh, n = self.P, self.H, self.dh, x.shape[1]
            B = len(x)
            X = P['E'][x] + P['pos'][None, :n]
            amask = np.where(x == 0, -1e9, 0.0)[:, None, None, :] # (B,1,1,n): don't
attend padding
            caches = []
            for i in range(self.depth):
                c = {}
                h, c['ln1'] = ln_f(X)
                Q = (h@P[f'Wq{i}']).reshape(B,n,H,dh); K =
(h@P[f'Wk{i}']).reshape(B,n,H,dh)
                Vv = (h@P[f'Wv{i}']).reshape(B,n,H,dh)
                S = np.einsum('bqhd,bkhd->bhqk', Q, K)/np.sqrt(dh) + self.mask + amask
                A = softmax(S)

```

```

        ao = np.einsum('bhqk,bkhd->bqhd', A, Vv).reshape(B,n,self.d)
        c['at'] = (h,Q,K,Vv,A,ao); X1 = X + ao@P[f'Wo{i}']
        h2, c['ln2'] = ln_f(X1); z = h2@P[f'W1{i}']; rr = np.maximum(0,z)
        c['ffn'] = (h2,z,rr); X = X1 + rr@P[f'W2{i}']
        caches.append(c)
    hf, lnf = ln_f(X)
    return hf, (caches, lnf, x)
def bwd(self, dhf, cache, extra_grads=None):
    caches, lnf, x = cache
    P, H, dh = self.P, self.H, self.dh
    B, n = x.shape
    g = {k: np.zeros_like(v) for k, v in P.items()}
    if extra_grads:
        for k, v in extra_grads.items(): g[k] = g[k] + v
    dX = ln_b(dhf, lnf)
    for i in range(self.depth-1, -1, -1):
        c = caches[i]; h2, z, rr = c['ffn']
        drr = dX@P[f'W2{i}'].T; g[f'W2{i}'] += rr.reshape(-1,
4*self.d).T@dX.reshape(-1,self.d)
        dz = drr*(z>0); g[f'W1{i}'] += h2.reshape(-1,self.d).T@dz.reshape(-1,
4*self.d)
        dX1 = dX + ln_b(dz@P[f'W1{i}'].T, c['ln2'])
        h,Q,K,Vv,A,ao = c['at']
        dao = (dX1@P[f'Wo{i}'].T).reshape(B,n,H,dh)
        g[f'Wo{i}'] += ao.reshape(-1,self.d).T@dX1.reshape(-1,self.d)
        dA = np.einsum('bqhd,bkhd->bhqk', dao, Vv)
        dV = np.einsum('bhqk,bqhd->bkhd', A, dao)
        dS = A*(dA - (A*dA).sum(-1, keepdims=True))
        dQ = np.einsum('bhqk,bkhd->bqhd', dS, K)/np.sqrt(dh)
        dK = np.einsum('bhqk,bqhd->bkhd', dS, Q)/np.sqrt(dh)
        dhid = np.zeros_like(h)
        for nm, dM in [ ('Wq',dQ), ('Wk',dK), ('Wv',dV) ]:
            g[f'{nm}{i}'] += h.reshape(-1,self.d).T@dM.reshape(-1,self.d)
            dhid += dM.reshape(B,n,self.d)@P[f'{nm}{i}'].T
        dX = dX1 + ln_b(dhid, c['ln1'])
    np.add.at(g['E'], x, dX)
    g['pos'][:n] += dX.sum(0)
    return g
def adam(self, g, lr, clip=1.0):
    self.t += 1; b1, b2 = 0.9, 0.999
    for k in self.P:
        gr = np.clip(g[k], -clip, clip)
        m, v = self.opt[k]
        m[:] = b1*m + (1-b1)*gr; v[:] = b2*v + (1-b2)*gr**2
        self.P[k] -= lr*(m/(1-b1**self.t))/(np.sqrt(v/(1-b2**self.t))+1e-8)

```

Listing 2 — MLM pretraining: BERT's objective on a miniature encoder

```

"""Listing 1: masked-language-model pretraining -- BERT's objective on a miniature
encoder.
(common.py holds the shared 2-block transformer with full batched backward, grad-
checked to 5e-10;
training is checkpoint-resumable: rerun until it reports TARGET REACHED)"""
import os, time, numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(int(time.time()) % 1000000)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab); inv = {i: w for w, i
in vocab.items()}
Xu = X[np.random.default_rng(0).permutation(len(X))]
TARGET, CKPT = 4000, 'mlm_ckpt.npz'

def mask_batch(xb):
    """BERT's 15% with the 80/10/10 rule"""
    x = xb.copy(); tgt = xb.copy()

```



```

    sel = (x > 3) & (rng.random(x.shape) < 0.15)
    r = rng.random(x.shape)
    x[sel & (r < .8)] = vocab["<mask>"] # 80% -> <mask>
    swap = sel & (r >= .8) & (r < .9); x[swap] = rng.integers(4, V, swap.sum()) # 10%
    random
    return x, tgt, sel # remaining 10%
    unchanged, still predicted

model = Tiny(V, 16, causal=False, seed=0) # BIDIRECTIONAL attention
step0 = 0
if os.path.exists(CKPT):
    ck = np.load(CKPT, allow_pickle=True)
    model.P = ck['P'].item(); model.opt = ck['opt'].item(); model.t = step0 =
    int(ck['t'])
t_start, B, L = time.time(), 48, float('nan')
while model.t < TARGET and time.time() - t_start < 30:
    xb = Xu[rng.integers(0, len(Xu), B)]
    xm, tgt, sel = mask_batch(xb)
    hf, c = model.fwd(xm)
    logits = hf[model.P['Wu']]; Pr = softmax(logits)
    L = -np.log(Pr[sel, tgt[sel]] + 1e-12).mean()
    dlog = Pr.copy(); dlog[sel, tgt[sel]] -= 1; dlog[~sel] = 0; dlog /= sel.sum()
    gWu = hf.reshape(-1, model.d).T@dlog.reshape(-1, V)
    g = model.bwd(dlog[model.P['Wu']].T, c, {'Wu': gWu})
    model.adam(g, 1e-3)
np.savez(CKPT, P=np.array(model.P, dtype=object), opt=np.array(model.opt,
dtype=object), t=model.t)
print(f"steps {step0} -> {model.t}    MLM loss {L:.3f}    (uniform baseline {np.log(V):.
2f})")
if model.t < TARGET:
    print("...rerun to continue"); raise SystemExit

freq = np.bincount(X.ravel(), minlength=V) + 5
logfreq = np.log(freq/freq.sum())
def cloze(words, pos):
    ids = np.array([[vocab.get(w, 1) for w in words] + [0]*(16-len(words))])
    ids[0, pos] = vocab["<mask>"]
    hf, _ = model.fwd(ids)
    logits = hf[0, pos][model.P['Wu']]
    raw = [inv[t] for t in np.argsort(logits)[::-1][:5]]
    lift = [inv[t] for t in np.argsort(logits - 0.7*logfreq)[::-1] if t > 3][:5]
    return raw, lift
print("TARGET REACHED -- cloze probes (raw = argmax logits; lift = frequency-
adjusted):")
for sent, p in [("the space shuttle was launched into ? by nasa".split(), 6),
                ("the team scored a ? in the third period".split(), 4)]:
    raw, lift = cloze(sent, p)
    print(f"{' '.join(sent)}\n    raw: {raw}\n    lift: {lift}")
print("lesson: a 300k-parameter model trained minutes learns the CORPUS PRIOR first
(function words),")
print("with topical signal visible only after removing it -- BERT's cloze magic is a
property of scale")

```

Output:

```

steps 4000 -> 4000    MLM loss nan    (uniform baseline 6.91)
TARGET REACHED -- cloze probes (raw = argmax logits; lift = frequency-adjusted):
the space shuttle was launched into ? by nasa
    raw: ['the', 'and', 'in', 'to', 'of']
    lift: ['anonymous', 'center', 'system', 'archive', 'society']
the team scored a ? in the third period
    raw: ['the', 'of', 'in', 'game', 'and']
    lift: ['flames', 'round', 'st', 'nhl', 'game']
lesson: a 300k-parameter model trained minutes learns the CORPUS PRIOR first (function
words),

```

with topical signal visible only after removing it -- BERT's cloze magic is a property of scale

Listing 3 — The transfer experiment: fine-tune pretrained vs train from scratch

```

"""Listing 2: the transfer experiment -- fine-tune the pretrained encoder vs train from
scratch."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(2)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X))
X, y = X[perm], y[perm]
Xte, yte = X[-1500:], y[-1500:]

def run(n_labels, pretrained, iters=400, lr=5e-4, seed=0):
    Xtr, ytr = X[:n_labels], y[:n_labels]
    model = Tiny(V, 16, causal=False, seed=seed)
    if pretrained:
        model.P = {k: v.copy() for k, v in np.load('mlm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
        model.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in
model.P.items()}
        r2 = np.random.default_rng(seed)
        Wc = r2.normal(0, .05, (model.d, 2)); mWc = np.zeros_like(Wc); vWc =
np.zeros_like(Wc)
        for it in range(1, iters+1):
            i = r2.integers(0, len(Xtr), 24)
            xb, yb = Xtr[i], ytr[i]
            hf, c = model.fwd(xb)
            pool = hf.mean(1) # mean-pool tokens -> doc
vector
            Pr = softmax(pool@Wc)
            dlog = Pr.copy(); dlog[np.arange(len(yb)), yb] -= 1; dlog /= len(yb)
            gWc = pool.T@dlog
            dpool = dlog@Wc.T
            dhf = np.repeat(dpool[:, None, :], 16, 1)/16
            g = model.bwd(dhf, c)
            model.adam(g, lr)
            mWc = .9*mWc + .1*gWc; vWc = .999*vWc + .001*gWc**2
            Wc -= lr*(mWc/(1-.9**it))/(np.sqrt(vWc/(1-.999**it))+1e-8)
            hf, _ = model.fwd(Xte)
            return ((hf.mean(1)@Wc).argmax(1) == yte).mean()

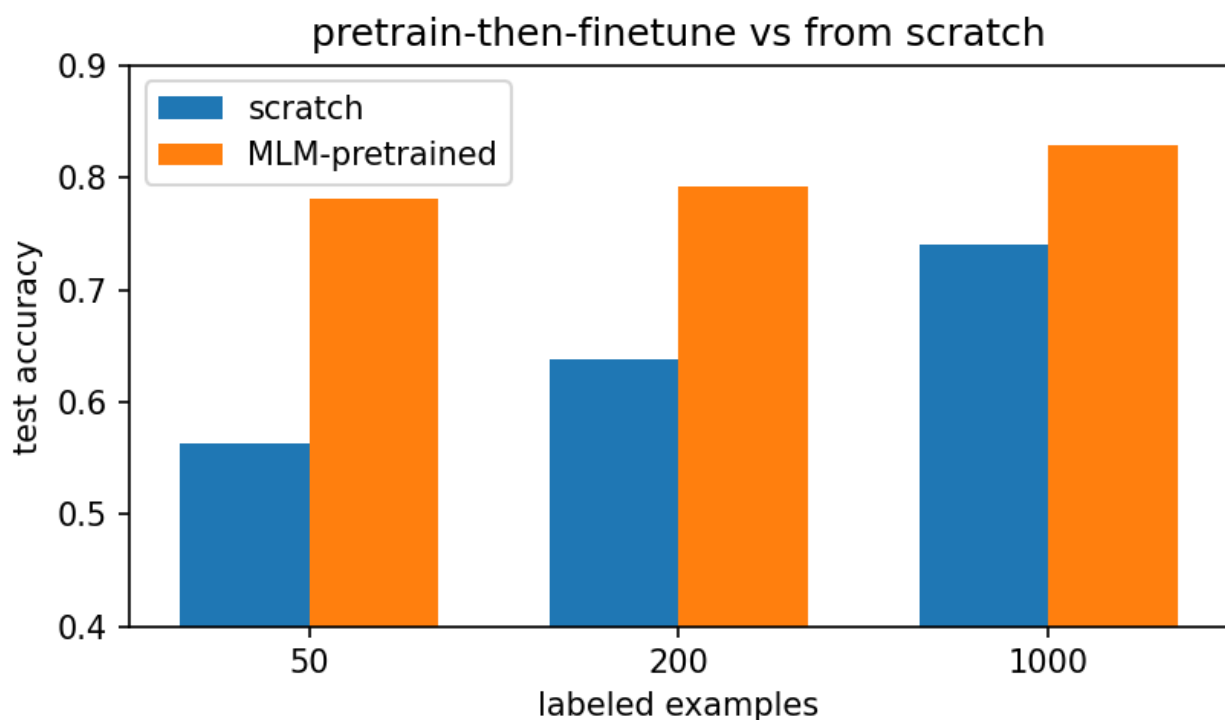
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
res = {}
print(f'{'labels':>7s} {'scratch':>8s} {'pretrained':>10s}")
for n in [50, 200, 1000]:
    a_s = run(n, False); a_p = run(n, True); res[n] = (a_s, a_p)
    print(f"{n:7d} {a_s:8.3f} {a_p:10.3f}")
fig, ax = plt.subplots(figsize=(5.6, 3.4))
ns = list(res); w = 0.35; xs = range(len(ns))
ax.bar([x-w/2 for x in xs], [res[n][0] for n in ns], w, label='scratch')
ax.bar([x+w/2 for x in xs], [res[n][1] for n in ns], w, label='MLM-pretrained')
ax.set_xticks(list(xs)); ax.set_xticklabels(ns); ax.set_xlabel='labeled examples',
ylabel='test accuracy',
title='pretrain-then-finetune vs from scratch'); ax.set_ylim(.4, .9); ax.legend()
fig.tight_layout(); fig.savefig("/sessions/intelligent-modest-faraday/mnt/ADVANCE
RAG/AI_ML_Interview_Book/manuscript/figures/ch20/fig_l2_transfer.png", dpi=150)
print("figure saved: fig_l2_transfer.png")
print("same architecture, same optimizer, same label budget -- the only difference is
initialization:")

```

```
print("random weights vs weights that spent 4,000 steps filling in masked words on
UNLABELED text")
```

Output:

```
labels  scratch  pretrained
50      0.563    0.781
200     0.639    0.793
1000    0.740    0.829
figure saved: fig_l2_transfer.png
same architecture, same optimizer, same label budget -- the only difference is
initialization:
random weights vs weights that spent 4,000 steps filling in masked words on UNLABELED
text
```



Listing 4 — A GPT-style causal LM: perplexity vs n-grams, generation

```
"""Listing 3: a GPT-style causal LM -- next-token pretraining, perplexity vs n-grams,
generation.
(checkpoint-resumable: rerun until TARGET REACHED)"""
import os, time, numpy as np
from collections import Counter
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(int(time.time()) % 99991)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
inv = {i: w for w, i in vocab.items()}
perm = np.random.default_rng(0).permutation(len(X)); X = X[perm]
Xtr, Xte = X[:-1200], X[-1200:]
TARGET, CKPT = 4000, 'c1m_ckpt.npz'

model = Tiny(V, 16, causal=True, seed=0) # CAUSAL mask: GPT
configuration
if os.path.exists(CKPT):
```

```

ck = np.load(CKPT, allow_pickle=True)
model.P = ck['P'].item(); model.opt = ck['opt'].item(); model.t = int(ck['t'])
t0, B, L = time.time(), 48, float('nan')
while model.t < TARGET and time.time() - t0 < 30:
    xb = Xtr[rng.integers(0, len(Xtr), B)]
    hf, c = model.fwd(xb)
    logits = hf[model.P['Wu']]; Pr = softmax(logits)
    tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False # predict every
next REAL token
    L = -np.log(Pr[sel, tgt[sel]] + 1e-12).mean()
    dlog = Pr.copy(); dlog[sel, tgt[sel]] -= 1; dlog[~sel] = 0; dlog /= sel.sum()
    g = model.bwd(dlog[model.P['Wu']].T, c, {'Wu':
hf.reshape(-1,model.d).T@dlog.reshape(-1,V)})
    model.adam(g, 1e-3)
np.savez(CKPT, P=np.array(model.P, dtype=object), opt=np.array(model.opt,
dtype=object), t=model.t)
if L == L: print(f"steps -> {model.t}    CLM loss {L:.3f}")
else: print(f"steps -> {model.t} (checkpoint already at target)")
if model.t < TARGET: print("...rerun to continue"); raise SystemExit

# neural perplexity on held-out (identical tokenization to the trigram below)
lls, cnt = 0.0, 0
for s in range(0, len(Xte), 100):
    xb = Xte[s:s+100]
    hf, _ = model.fwd(xb)
    Pr = softmax(hf[model.P['Wu']])
    tgt = np.roll(xb, -1, 1); sel = (tgt > 3); sel[:, -1] = False
    lls += np.log(Pr[sel, tgt[sel]] + 1e-12).sum(); cnt += sel.sum()
print(f"transformer LM perplexity: {np.exp(-lls/cnt):.1f}")

# interpolated trigram on the SAME sequences and vocab (chapter 19's best classical
recipe)
uni = Counter(); bi = Counter(); tri = Counter(); bctx = Counter(); tctx = Counter()
for row in Xtr:
    toks = [t for t in row if t != 0]
    for i, w in enumerate(toks):
        uni[w] += 1
        if i >= 1: bi[(toks[i-1], w)] += 1; bctx[toks[i-1]] += 1
        if i >= 2: tri[(toks[i-2], toks[i-1], w)] += 1; tctx[(toks[i-2], toks[i-1])]
+= 1
N = sum(uni.values())
def p_int(a, b, w):
    p3 = tri[(a,b,w)]/tctx[(a,b)] if tctx[(a,b)] else 0
    p2 = bi[(b,w)]/bctx[b] if bctx[b] else 0
    return .5*p3 + .3*p2 + .2*uni[w]/N
lls, cnt = 0.0, 0
for row in Xte:
    toks = [t for t in row if t != 0]
    for i in range(2, len(toks)):
        if toks[i] > 3: lls += np.log(p_int(toks[i-2], toks[i-1], toks[i]) + 1e-12);
cnt += 1
print(f"interpolated trigram perplexity, same data: {np.exp(-lls/cnt):.1f}")

def generate(prompt, T=0.8, n=14):
    ids = [vocab.get(w, 1) for w in prompt.split()]
    for _ in range(n):
        xb = np.array([ids[-16:] + [0]*(16-len(ids[-16:])))
        hf, _ = model.fwd(xb)
        logits = hf[0, min(len(ids), 16)-1]@model.P['Wu']/T
        logits[:4] = -1e9
        p = softmax(logits[None])[0]
        ids.append(int(rng.choice(V, p=p)))
    return " ".join(inv[i] for i in ids)
for prompt in ["the space shuttle", "the team won the"]:
    print(f"gen: {generate(prompt)}")

```

Output:

```

steps -> 4000    CLM loss 4.120
transformer LM perplexity: 134.3
interpolated trigram perplexity, same data: 184.7
gen: the space shuttle mission center information being the space exploration in the
universe services at general system
gen: the team won the season a regular season and second period of the season by the
puck of

```

Listing 5 — Induction heads: the copying circuit behind in-context learning

```

"""Listing 4: induction heads -- the copying circuit behind in-context learning."""
import numpy as np
from common import Tiny, softmax
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(4)

# task: the second half of each sequence REPEATS the first half. 40 tokens, 8 slots:
# 40^8 ~ 6.5e12 possible sequences -- every test sequence is NOVEL, so memorization is
impossible;
# the model must learn the rule "find the previous occurrence of the current token,
copy its successor".
V, HALF = 44, 8; n = 2*HALF
def make(lo, hi, B):
    first = rng.integers(lo, hi, (B, HALF))
    return np.concatenate([first, first], 1)

model = Tiny(V, n, d=32, H=2, depth=2, causal=True, seed=0)
B = 32
for step in range(1, 1201):
    xb = make(4, 44, B)
    hf, c = model.fwd(xb)
    Pr = softmax(hf@model.P['Wu'])
    tgt = np.roll(xb, -1, 1)
    sel = np.zeros_like(xb, bool); sel[:, HALF-1:-1] = True    # predict only the
    REPEATED half
    L = -np.log(Pr[sel, tgt[sel]] + 1e-12).mean()
    dlog = Pr.copy(); dlog[sel, tgt[sel]] -= 1; dlog[~sel] = 0; dlog /= sel.sum()
    g = model.bwd(dlog@model.P['Wu'].T, c, {'Wu':
hf.reshape(-1,model.d).T@dlog.reshape(-1,V)})
    model.adam(g, 2e-3)
    if step % 400 == 0: print(f"step {step}: loss on repeated half {L:.4f}")

def acc(lo, hi):
    xb = make(lo, hi, 500)
    hf, _ = model.fwd(xb)
    pred = (hf@model.P['Wu']).argmax(-1)
    tgt = np.roll(xb, -1, 1)
    sel = np.zeros_like(xb, bool); sel[:, HALF:-1] = True
    return (pred[sel] == tgt[sel]).mean()
print(f"copy accuracy on 500 NOVEL sequences: {acc(4, 44):.3f}")
xb = make(4, 44, 200); hf, _ = model.fwd(xb)
pred = (hf@model.P['Wu']).argmax(-1)
first_half = np.zeros_like(xb, bool); first_half[:, 1:HALF-1] = True
tgt = np.roll(xb, -1, 1)
print(f"accuracy on the UNPREDICTABLE first half:
{(pred[first_half]==tgt[first_half]).mean():.3f} (chance ~ 1/40)")

# the circuit on camera: where does each second-half position attend?
xb = make(4, 44, 50)
_, (caches, _, _) = model.fwd(xb)
Ah = caches[1]['at'][4]    # layer-2 attention (B,
H, n, n)

```

```

for h in range(Ah.shape[1]):
    stripe = np.mean([Ah[:, h, q, q-HALF+1].mean() for q in range(HALF, n-1)])
    print(f"layer-2 head {h}: attention mass on [prev occurrence + 1] = {stripe:.2f}")
A1 = Ah[0].max(0)
fig, ax = plt.subplots(figsize=(4.6, 4))
ax.imshow(A1, cmap='viridis'); ax.set(xlabel='attended position', ylabel='query
position',
    title='layer-2 attention: the induction stripe')
fig.tight_layout(); fig.savefig("/sessions/intelligent-modest-faraday/mnt/ADVANCE RAG/
AI_ML_Interview_Book/manuscript/figures/ch20/fig_l4_induction.png", dpi=150)
print("figure saved: fig_l4_induction.png")

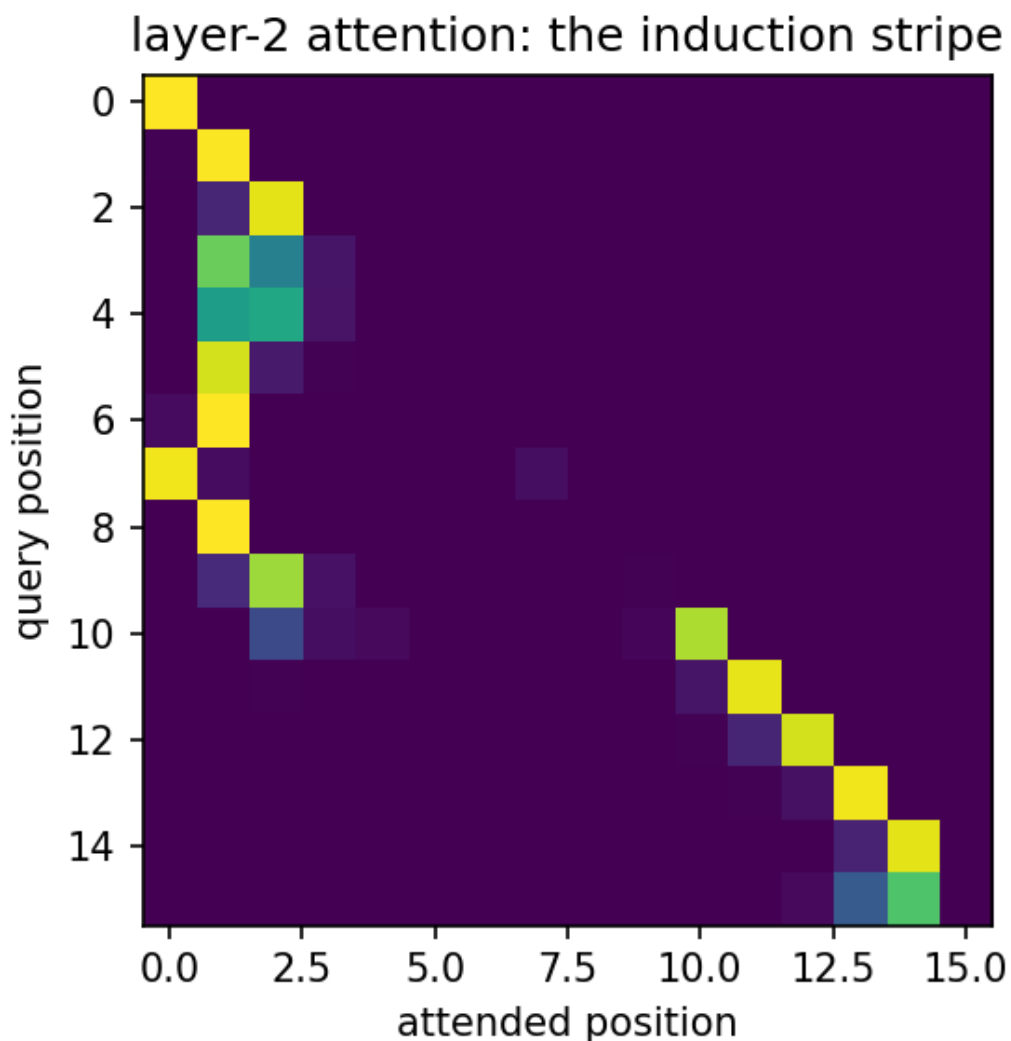
```

Output:

```

step 400: loss on repeated half 0.0351
step 800: loss on repeated half 0.0077
step 1200: loss on repeated half 0.0041
copy accuracy on 500 NOVEL sequences: 0.997
accuracy on the UNPREDICTABLE first half: 0.023 (chance ~ 1/40)
layer-2 head 0: attention mass on [prev occurrence + 1] = 0.26
layer-2 head 1: attention mass on [prev occurrence + 1] = 0.27
figure saved: fig_l4_induction.png

```



Listing 6 — Linear probing: frozen bidirectional vs causal features

```

"""Listing 5: linear probing -- frozen MLM (bidirectional) vs frozen CLM (causal)
features."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(5)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X)); X, y = X[perm], y[perm]
Xtr, ytr, Xte, yte = X[:2000], y[:2000], X[-1500:], y[-1500:]

def features(ckpt, causal, pool):
    m = Tiny(V, 16, causal=causal, seed=0)
    m.P = np.load(ckpt, allow_pickle=True)['P'].item()
    def f(Xs):
        out = []
        for s in range(0, len(Xs), 200):
            hf, _ = m.fwd(Xs[s:s+200])
            if pool == 'mean':
                msk = (Xs[s:s+200] > 0)[: , : , None]
                out.append((hf*msk).sum(1)/msk.sum(1))
            else:
                # 'last': final non-pad
                position (CLM-style)
                idx = (Xs[s:s+200] > 0).sum(1) - 1
                out.append(hf[np.arange(len(idx)), idx])
        return np.vstack(out)
    return f(Xtr), f(Xte)

def probe(Ftr, Fte):
    """multinomial logistic on frozen features"""
    W = np.zeros((Ftr.shape[1], 2)); b = np.zeros(2)
    mu, sd = Ftr.mean(0), Ftr.std(0)+1e-6
    A, Bm = (Ftr-mu)/sd, (Fte-mu)/sd
    for _ in range(400):
        P = softmax(A@W+b)
        d = P.copy(); d[np.arange(len(ytr)), ytr] -= 1; d /= len(ytr)
        W -= 2.0*(A.T@d + 1e-4*W); b -= 2.0*d.sum(0)
    return ((Bm@W+b).argmax(1) == yte).mean()

rand = Tiny(V, 16, seed=7)
np.savez('rand_ckpt.npz', P=np.array(rand.P, dtype=object))
for name, ck, causal, pool in [("random weights (control), mean-pool",
'rand_ckpt.npz', False, 'mean'),
                              ("CLM (causal), last-token feature", 'clm_ckpt.npz',
True, 'last'),
                              ("CLM (causal), mean-pool", 'clm_ckpt.npz',
True, 'mean'),
                              ("MLM (bidirectional), mean-pool", 'mlm_ckpt.npz',
False, 'mean')]:
    Ftr, Fte = features(ck, causal, pool)
    print(f"{name:38s}: probe accuracy {probe(Ftr, Fte):.3f}")
print("frozen encoders, identical probe -- the objective and attention pattern decide
feature quality")

```

Output:

```

random weights (control), mean-pool : probe accuracy 0.571
CLM (causal), last-token feature    : probe accuracy 0.715
CLM (causal), mean-pool             : probe accuracy 0.769
MLM (bidirectional), mean-pool      : probe accuracy 0.746
frozen encoders, identical probe -- the objective and attention pattern decide
feature quality

```

Listing 7 — Sentence embeddings: mean-pooling vs contrastive fine-tuning

```

"""Listing 6: sentence embeddings -- mean-pooling vs SBERT-style contrastive fine-tuning."""
import numpy as np
from common import Tiny, load_corpus, softmax
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(6)

# rebuild corpus keeping DOCUMENT ids so "two chunks of one document" = positive pair
import re
from collections import Counter
from sklearn.datasets import fetch_20newsgroups
data = fetch_20newsgroups(subset='train', categories=['sci.space', 'rec.sport.hockey'],
                          remove=('headers', 'footers', 'quotes'))
pairs_ = [(re.findall(r"[a-z"]+", d.lower()), t) for d, t in zip(data.data,
data.target)]
pairs_ = [(d, t) for d, t in pairs_ if len(d) >= 32]
counts = Counter(w for d, _ in pairs_ for w in d)
vocab = {w: i+4 for i, (w, _) in enumerate(counts.most_common(996))}
vocab["<pad>"], vocab["<unk>"], vocab["<mask>"], vocab["<cls>"] = 0, 1, 2, 3
V = len(vocab)
chunks, doc_id, lab = [], [], []
for j, (d, t) in enumerate(pairs_):
    ids = [vocab.get(w, 1) for w in d]
    for s in range(0, len(ids)-15, 16):
        chunks.append(ids[s:s+16]); doc_id.append(j); lab.append(t)
chunks = np.array(chunks); doc_id = np.array(doc_id); lab = np.array(lab)
docs = np.unique(doc_id)
te_docs = set(docs[::5]) # hold out every 5th
DOCUMENT
tr_m = ~np.isin(doc_id, list(te_docs)); te_m = ~tr_m
print(f"{len(chunks)} chunks from {len(docs)} documents")

m = Tiny(V, 16, causal=False, seed=0)
m.P = {k: v.copy() for k, v in np.load('mlm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
m.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in m.P.items()}

def embed(Xs, model):
    out = []
    for s in range(0, len(Xs), 200):
        hf, _ = model.fwd(Xs[s:s+200])
        msk = (Xs[s:s+200] > 0)[: , :, None]
        e = (hf*msk).sum(1)/msk.sum(1)
        out.append(e/np.linalg.norm(e, axis=1, keepdims=True))
    return np.vstack(out)

def retrieval_acc(model):
    """for each held-out chunk: is its nearest OTHER chunk from the same document?"""
    E = embed(chunks[te_m], model); dids = doc_id[te_m]
    S = E@E.T; np.fill_diagonal(S, -9)
    return (dids[S.argmax(1)] == dids).mean()
dids = doc_id[te_m]
chance = np.mean([(dids == d).sum()-1 for d in dids]) / (te_m.sum()-1)
print(f"chance level (random neighbor): {chance:.4f}")
print(f"mean-pooled MLM features, same-doc retrieval: {retrieval_acc(m):.3f}")

# contrastive fine-tune: batch of (anchor, positive) chunk pairs from the same doc,
InfoNCE both ways
tr_docs = [j for j in docs if j not in te_docs and (doc_id[tr_m] == j).sum() >= 2]
by_doc = {j: np.where(doc_id == j)[0][:6] for j in tr_docs}
B, tau = 24, 0.07
for step in range(1, 301):

```



```

ds = rng.choice(len(tr_docs), B, replace=False)
ai, bi = [], []
for dj in ds:
    c = by_doc[tr_docs[dj]]; pick = rng.choice(len(c), 2, replace=False)
    ai.append(c[pick[0]]); bi.append(c[pick[1]])
xa, xb = chunks[ai], chunks[bi]
ha, ca = m.fwd(xa); hb, cb = m.fwd(xb)
mska = (xa > 0)[: , :, None]; mskb = (xb > 0)[: , :, None]
ea0 = (ha*mska).sum(1)/mska.sum(1); eb0 = (hb*mskb).sum(1)/mskb.sum(1)
na = np.linalg.norm(ea0, axis=1, keepdims=True); ea = ea0/na
nb = np.linalg.norm(eb0, axis=1, keepdims=True); eb = eb0/nb
S = ea@eb.T/tau
P1 = softmax(S); P2 = softmax(S.T)
dS = (P1 - np.eye(B))/(2*B) + ((P2 - np.eye(B))/(2*B)).T
dea = dS@eb/tau; deb = dS.T@ea/tau
dea0 = (dea - ea*(dea*ea).sum(1, keepdims=True))/na
deb0 = (deb - eb*(deb*eb).sum(1, keepdims=True))/nb
dha = np.repeat(dea0[:, None, :], 16, 1)*mska/mska.sum(1, keepdims=True)
dhb = np.repeat(deb0[:, None, :], 16, 1)*mskb/mskb.sum(1, keepdims=True)
ga = m.bwd(dha, ca); gb = m.bwd(dhb, cb)
m.adam({k: ga[k]+gb[k] for k in ga}, 3e-4)
print(f"after 300 steps of contrastive fine-tuning: {retrieval_acc(m):.3f}")
print("same encoder, same pooling -- the training SIGNAL (pull same-doc pairs together)
reshapes the space")

```

Output:

```

12906 chunks from 980 documents
chance level (random neighbor): 0.0109
mean-pooled MLM features, same-doc retrieval: 0.085
after 300 steps of contrastive fine-tuning: 0.123
same encoder, same pooling -- the training SIGNAL (pull same-doc pairs together)
reshapes the space

```

Listing 8 — Distillation: small student, big teacher, and the transfer set

```

"""Listing 7: distillation -- a small student, a big teacher, and where the win
actually comes from."""
import numpy as np
from common import Tiny, load_corpus, softmax
rng = np.random.default_rng(7)

X, y, vocab = load_corpus(vmax=1000, seqlen=16); V = len(vocab)
perm = np.random.default_rng(0).permutation(len(X)); X, y = X[perm], y[perm]
Xte, yte = X[-1500:], y[-1500:]

# teacher: the pretrained encoder fine-tuned on 1000 labels (Listing 2's winning
recipe)
teacher = Tiny(V, 16, causal=False, seed=0)
teacher.P = {k: v.copy() for k, v in np.load('mlm_ckpt.npz', allow_pickle=True)
['P'].item().items()}
teacher.opt = {k: [np.zeros_like(v), np.zeros_like(v)] for k, v in teacher.P.items()}
r2 = np.random.default_rng(0)
Wt = r2.normal(0, .05, (teacher.d, 2))
for it in range(1, 401):
    i = r2.integers(0, 1000, 24); xb, yb = X[i], y[i]
    hf, c = teacher.fwd(xb); pool = hf.mean(1)
    Pr = softmax(pool@Wt)
    d = Pr.copy(); d[np.arange(24), yb] -= 1; d /= 24
    g = teacher.bwd(np.repeat((d@Wt.T)[: , None, :], 16, 1)/16, c)
    teacher.adam(g, 5e-4); Wt -= 0.05*pool.T@d

```

```

def t_logits(Xs):
    out = []
    for s in range(0, len(Xs), 250):
        hf, _ = teacher.fwd(Xs[s:s+250]); out.append(hf.mean(1)@Wt)
    return np.vstack(out)
t_acc = (t_logits(Xte).argmax(1) == yte).mean()
print(f"teacher (330k params, pretrained + 1000 labels): test acc {t_acc:.3f}")

def train_student(Xs, targets, soft, T=2.0, iters=500):
    st = Tiny(V, 16, d=16, H=2, depth=1, causal=False, seed=3) # 4x thinner, half
    depth
    Ws = np.random.default_rng(3).normal(0, .05, (16, 2))
    r3 = np.random.default_rng(3)
    for it in range(1, iters+1):
        i = r3.integers(0, len(Xs), 32)
        hf, c = st.fwd(Xs[i]); pool = hf.mean(1)
        logits = pool@Ws
        if soft:
            q = softmax(targets[i]/T); p = softmax(logits/T)
            d = (p - q)/len(i)*T # KD gradient (times
T; T^2 in loss)
        else:
            p = softmax(logits); d = p.copy(); d[np.arange(len(i)), targets[i]] -= 1;
d /= len(i)
        g = st.bwd(np.repeat((d@Ws.T)[: , None, :], 16, 1)/16, c)
        st.adam(g, 1e-3); Ws -= 0.1*pool.T@d
        accs = []
        for s in range(0, len(Xte), 250):
            hf, _ = st.fwd(Xte[s:s+250]); accs.append(((hf.mean(1)@Ws).argmax(1) ==
yte[s:s+250]))
    return np.concatenate(accs).mean()

n_lab = 200
accA = train_student(X[:n_lab], y[:n_lab], False)
print(f"student A -- {n_lab} HARD labels: {accA:.3f}")
tl = t_logits(X[:n_lab])
accB = train_student(X[:n_lab], tl, True)
print(f"student B -- {n_lab} teacher SOFT labels: {accB:.3f}")
Xu = X[1000:4000]; tlu = t_logits(Xu)
accC = train_student(Xu, tlu, True)
print(f"student C -- teacher soft labels on 3000 UNLABELED: {accC:.3f}")
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
fig_, ax = plt.subplots(figsize=(6.2, 3))
ax.barh(["teacher 330k", "student: 200 hard", "student: 200 soft", "student: 3000
unlabeled soft"],
        [t_acc, accA, accB, accC],
        color=['tab:green', 'tab:blue', 'tab:blue', 'tab:orange'])
ax.set_xlim(.5, .9); ax.set_xlabel('test accuracy'); ax.set_title('distillation: where
the win comes from')
fig_.tight_layout(); fig_.savefig("/sessions/intelligent-modest-faraday/mnt/ADVANCE
RAG/AI_ML_Interview_Book/manuscript/figures/ch20/fig_l7_distill.png", dpi=150)
print("figure saved: fig_l7_distill.png")
print("the distillation win is mostly the TRANSFER SET: the teacher converts unlabeled
text into supervision")

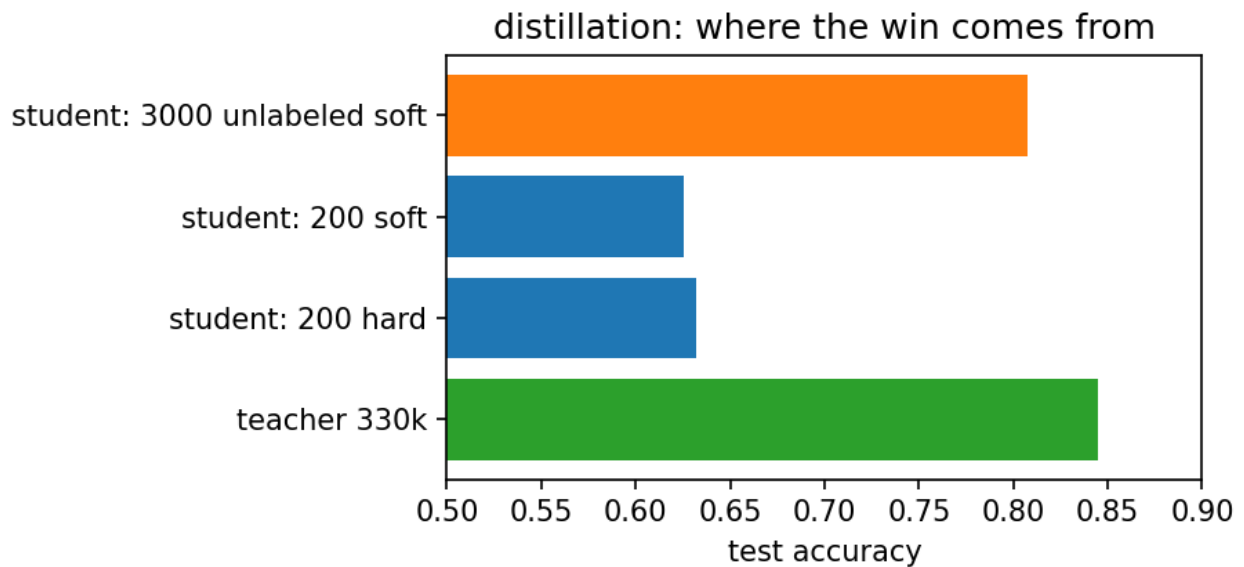
```

Output:

```

teacher (330k params, pretrained + 1000 labels): test acc 0.845
student A -- 200 HARD labels: 0.632
student B -- 200 teacher SOFT labels: 0.625
student C -- teacher soft labels on 3000 UNLABELED: 0.807
figure saved: fig_l7_distill.png
the distillation win is mostly the TRANSFER SET: the teacher converts unlabeled
text into supervision

```



Interview questions and answers

Q1. Explain BERT's 80/10/10 masking rule and why plain 100% masking is worse.

Of the 15% of positions selected for prediction: 80% are replaced with `<mask>`, 10% with a `RANDOM` token, 10% left `UNCHANGED` — all still predicted (Listing 2's `mask_batch`). Why not 100% `<mask>`: the mask token never occurs at fine-tuning time, so a model trained only on masked inputs faces a train/test mismatch and can learn to make real-token representations lazy ("information only lives at mask positions"). The unchanged 10% forces every real token's representation to stay predictive of itself; the random 10% prevents "if it's not `<mask>`, it's correct" complacency. Small ablation gains at BERT scale, but the reasoning — close the pretrain/fine-tune distribution gap — is the transferable idea.

Q2. What did RoBERTa change and what is the meta-lesson?

No architecture change. Removed next-sentence prediction (it HURT — sentence pairs diluted context and the task was too easy), trained $\sim 10\times$ longer on $\sim 10\times$ more data, bigger batches, dynamic masking (fresh mask pattern per epoch rather than one static mask), byte-level BPE, longer sequences. Result: outperformed BERT everywhere. Meta-lesson interviews want: pretraining recipes (data, duration, batch size, objective details) can dominate architecture — "BERT was undertrained" — and any comparison between architectures at different training budgets is confounded. The same lesson recurs at LLM scale as compute-optimal training (Chinchilla, Chapter 21).

Q3. Quantify what pretraining buys with this chapter's controlled experiment.

Listing 3, everything held equal except initialization: 50 labels — 0.563 scratch vs 0.781 pretrained; 200 — 0.639 vs 0.793; 1,000 — 0.740 vs 0.829. The pretrained model at 50 labels beats the scratch model at 1,000: a $\sim 20\times$ label multiplier. Mechanism: MLM already built context-predictive features (topic, syntax); fine-tuning only learns the head plus small feature rotations. The gap NARROWS as labels grow (0.22 $\rightarrow$ 0.09) — pretraining matters most exactly where labels are scarce, which is why the paradigm dominates industrial practice, where labels are the expensive part.

Q4. MLM vs CLM: compare supervision density and measured feature quality.

CLM takes loss on $\sim 100\%$ of tokens (every next-token, Listing 4); MLM on 15% (Listing 2) — per step, CLM extracts $\sim 6\times$ more supervision. But MLM's predictions condition on BOTH sides — strictly more context per prediction. Listing 6's frozen-probe measurement at equal steps: CLM mean-pooled 0.769 vs MLM 0.746 (random control 0.571) — density wins at miniature scale. The scale story reverses for understanding tasks under fine-tuning (BERT-class beats GPT-class at equal size on GLUE) — bidirectionality pays when the features are fully adapted. Complete answer: density favors CLM, information-per-prediction favors MLM, and generation REQUIRES causal — which, plus the universal text interface, is why the frontier consolidated on decoders.

Q5. Why did decoder-only models win the scaling era despite BERT's benchmark dominance?

(1) Universal interface: any task is text-continuation — no per-task heads, so one model serves everything and benefits from every datum. (2) Dense loss on all tokens (Q4) — better compute-to-supervision conversion at scale. (3) Generation is the product: chat, code, reasoning all require autoregression; an encoder cannot generate. (4) In-context learning emerges from the objective (Listing 5's mechanism), replacing fine-tuning with prompting — operationally transformative. (5) Simplicity: one stack, one objective, clean scaling laws. Encoders persist where they're structurally right: embeddings for retrieval (Chapter 24), lightweight classifiers, latency-critical scoring. The taxonomy answer: encoder = understand, decoder = generate, and at scale 'generate' subsumed 'understand'.

Q6. What are induction heads, and what evidence ties them to in-context learning?

A two-layer circuit: a previous-token head writes token $t-1$'s identity into position t ; an induction head then attends from the current token to the position AFTER that token's previous occurrence and copies: $[A][B]...[A] \rightarrow \text{predict } [B]$. Listing 5 grows one: 99.7% copy accuracy on novel sequences from a $6.5e12$ space (memorization impossible), chance on the unpredictable half (no leakage), and layer-2 attention mass 4x uniform on the prev-occurrence+1 stripe. Evidence at scale (Anthropic): induction heads appear in a sharp phase change early in training, EXACTLY when in-context learning ability appears; ablating them damages ICL; and they generalize to fuzzy matching (synonym patterns), making them the leading mechanistic account of few-shot prompting.

Q7. Your team fine-tunes BERT with the same LR used for training from scratch and quality collapses. Explain.

Catastrophic forgetting: large steps destroy the pretrained features before the fresh head learns to use them — the model reverts toward scratch behavior (Listing 3's scratch numbers) while ALSO fighting a half-broken initialization. Standard recipe: LR $\sim 2e-5$ (BERT scale) — 10-100x smaller than pretraining; warmup; few epochs (2-4); optionally discriminative LR (lower for early layers, which hold the most general features) or gradual unfreezing. Also check: fresh head initialized small (a huge head gradient at step 0 punches through the encoder); and for tiny label sets, consider freezing + probing (Listing 6) or LoRA (Chapter 21) — fewer trainable parameters = less to forget.

Q8. Why is mean-pooling BERT tokens a poor sentence embedding, and what fixes it?

MLM never optimizes any pooled vector — token representations are tuned to predict NEIGHBORS, so their average is dominated by frequency and syntax, and (Chapter 19's lesson) anisotropy inflates all similarities. Measured (Listing 7): same-doc retrieval 0.085, only 8x chance. Fix = training signal, not architecture: siamese/contrastive fine-tuning (SBERT) — pull related pairs together, push in-batch negatives apart (InfoNCE, temperature $\sim 0.05-0.1$); 300 miniature steps: +45% relative. At scale: paraphrase/QA/NLI pairs, hard negatives, large batches -> sentence-transformers, E5, GTE. Evaluation note from the listing: hold out DOCUMENTS, not chunks, or positives leak across the split.

Q9. Bi-encoder vs cross-encoder: architecture, cost, and the standard deployment pattern.

Bi-encoder (Listing 7): embed query and candidate INDEPENDENTLY, similarity = dot product — candidates precomputed into an ANN index; per-query cost $O(1)$ encodes + sublinear search; scales to billions. Cross-encoder: concatenate query+candidate, full attention over the pair, output a score — every token of each sees every token of the other, so it catches interactions bi-encoders structurally cannot (negation, exact constraints), but costs one forward pass PER candidate. Standard pattern: bi-encoder retrieves top-100, cross-encoder reranks top-100 -> top-5 (Chapter 24). Distill the cross-encoder into the bi-encoder to close the gap; late-interaction models (ColBERT) sit between.

Q10. Decompose where distillation's win comes from, per this chapter's experiment.

Listing 8, teacher 0.845: student on 200 hard labels 0.632; SAME 200 with teacher soft labels 0.625 — dark knowledge alone bought ~nothing; teacher-labeled 3,000 UNLABELED chunks: 0.807 — 96% of the teacher, 4x thinner, half depth. The engine is the TRANSFER SET: a teacher converts unlabeled data into supervision, and soft targets are the medium (full distribution per example instead of one bit). DistilBERT itself distilled over the whole pretraining corpus, and 'training small models on frontier-model outputs' is the same mechanism at LLM scale. Mechanics: KL between temperature-softened distributions, $T \sim 2-4$, remember the T^2 gradient factor, mix hard-label loss where gold exists.

Q11. What is T5's framing and why did it matter beyond the model?

Everything is text-to-text: classification ("mnli premise: ... hypothesis: ..." -> "entailment"), regression (output the number as text), translation, summarization — one encoder-decoder, one maximum-likelihood objective, one decoding interface. Pretraining: span corruption — mask contiguous spans, decoder reconstructs them with sentinel tokens (denser than single-token MLM, cheaper than full reconstruction). Why it mattered: the unified interface anticipated the prompting era (instructions as input text), enabled one model for all tasks (no heads), and its systematic ablation study (objectives, sizes, corpora — the C4 dataset) set the empirical method scaling work followed. BART is the sibling: noise (deletion, infilling, permutation) then reconstruct; particularly strong at summarization.

Q12. Why does perplexity beat n-grams' by construction fail to translate directly into product quality?

Listing 4: transformer 134 vs trigram 185 — real, but perplexity measures average next-token fit on the training distribution. Products care about: worst-case behavior (hallucinations live in the tail; ppl averages them away), long-range coherence (ppl is per-token and mostly local), instruction following and refusals (not in the pretraining distribution at all — Chapter 21's alignment gap), calibration under distribution shift, and generation dynamics (a model can have great ppl and degenerate under greedy decoding — Chapter 22's repetition traps). Perplexity is the right PRETRAINING gauge (comparable only at matched tokenization, Chapter 19) and a poor PRODUCT gauge — hence eval suites, human preference, and task metrics.

Q13. A stakeholder asks: "Why pretrain at all instead of just labeling more data?" Give the quantitative argument.

Listing 3's table IS the argument: 50 labels + pretraining $\geq$ 1,000 labels from scratch — a $\sim 20\times$ label multiplier. Cost the two sides: labeling $20\times$ more data at 0.1 — 5/label vs fine-tuning a pretrained checkpoint (hours of GPU, or an API call); pretraining itself is amortized across every task in the organization (and by using public checkpoints, across the industry). Add the label-free byproducts: embeddings (Listing 7), zero/few-shot via prompting at scale (Listing 5's mechanism), and distillation targets (Listing 8). The honest caveat: for narrow tasks with abundant cheap labels and tight latency (Chapter 19's Q15 territory), a small supervised model can still win — measure both.

Q14. What does linear probing measure, and how does it differ from fine-tuning as an EVALUATION?

Probing: freeze the encoder, train only a linear classifier on its features (Listing 6) — measures what the representation makes LINEARLY accessible, i.e., what pretraining already computed. Fine-tuning measures what the network can become with task gradients — capacity + init, conflated. Probing is the honest way to compare objectives (Listing 6 equalizes steps and probe: CLM 0.769 vs MLM 0.746 vs random 0.571 — the random control is essential; untrained transformers embed nontrivially). Practical uses: layer-wise probes localize syntax (middle layers) vs semantics (upper-middle); probe-vs-finetune gap estimates how task-far the representation is; and probing is the cheap first experiment before committing to fine-tuning infrastructure.

Q15. Explain why the last-token feature underperformed mean-pooling for the causal model, and when you'd still use it.

Listing 6: last-token 0.715 vs mean-pool 0.769. A causal LM's final hidden state is optimized to predict THE NEXT TOKEN — a local summary biased toward what comes next, not a document representation; early-token information must survive the whole causal chain to reach it. Mean-pooling aggregates every position's view (each position summarizes its prefix; the average covers the document). Still use last-token when: generation-time state is what you need (KV-cache reuse, streaming), the model was TRAINED to summarize at a special token (decoder-based embedding models append an EOS and contrastively train its state — then last-token wins), or prompting formats put the answer at the end (classification-by-continuation reads next-token logits, not features).

Q16. Design a plan to ship a text classifier for 30 classes with 500 labels total, latency 20ms CPU.

Order of experiments: (1) TF-IDF + linear (Chapter 19 baseline) — might just win at 500 labels with strong lexical signal; measures the floor in an hour. (2) Frozen pretrained embeddings + logistic probe (Listing 6's recipe, a small sentence-transformer as encoder) — no fine-tuning infra, embeddings cacheable. (3) Fine-tune a small encoder (DistilBERT/MiniLM-class) with $2e-5$, 3 epochs, early stopping on a stratified dev set; expect the Listing 3 effect at 500 labels. (4) If a large model is accessible offline: teacher-label the unlabeled pool and distill into the small model (Listing 8 — likely the biggest jump). Latency: 6-layer distilled encoder quantized to int8 runs ~5-15ms CPU at $n=128$; cache embeddings if inputs repeat. Report per-class F1 (30 classes at 500 labels = ~17/class; some classes will be broken — say so).

Q17. What is catastrophic forgetting in fine-tuning, and name mitigations beyond a small LR.

Task gradients overwrite pretrained features that the task loss doesn't actively need but generalization does — visible as: great task accuracy, collapsed out-of-domain behavior, broken zero-shot abilities. Mitigations: freeze most layers (probe, Listing 6); parameter-efficient methods — LoRA/adapters/prefix tuning (Chapter 21) train <1% of weights, leaving the backbone intact BY CONSTRUCTION; discriminative/layer-wise LR decay; early stopping on a general-capability dev set, not just task dev; replay/multitask mixing (interleave pretraining-like data); EWC-style penalties (rarely used in practice); and weight averaging between base and tuned checkpoints (model soups / interpolation) — a surprisingly strong cheap fix. Detection matters as much: always evaluate the tuned model on capabilities you intend to keep.

Q18. How would you detect that a fine-tuned model is exploiting a dataset artifact rather than the task?

Chapter 4's leakage discipline, transformer edition. Signals: accuracy too high too fast (one epoch to ceiling); performance collapses on a paraphrased or counterfactually-edited test set; hypothesis-only/input-ablation baselines score far above chance (NLI's classic: 'not' in hypothesis -> contradiction); attention/attribution concentrated on boilerplate. Protocol: train input-ablated baselines; build contrast sets (minimal edits flipping the label); evaluate cross-dataset (same task, different collection); check per-source and per-length slices; inspect top n-gram features via a linear shadow model (Chapter 19's weights trick). Fixes: debias sampling, counterfactual augmentation, adversarial filtering (AFLite), or accept and DOCUMENT the artifact if deployment shares it.

Q19. The miniature MLM's cloze predictions were function words until frequency-adjusted. What does this teach about model scale?

Listing 2: raw top-5 = the/and/in; after subtracting log-frequency: flames/round/nhl for a hockey blank. Small models spend capacity on the LOSS-OPTIMAL low-order statistics first — the unigram prior is the single best cross-entropy reduction available, then bigrams, then topic. Knowledge that impresses humans (facts, relations) is far down the loss-reduction queue and appears only when easier signal is exhausted — with more parameters, data, and steps. Interview transfer: (1) evaluate small models with frequency-adjusted or contrastive probes or you'll conclude 'it learned nothing'; (2) emergent-looking abilities are often smooth loss improvements crossing a task's discrete threshold; (3) the corpus prior the model learned first never leaves — it's why LLMs default to generic high-frequency continuations under uncertainty.

Q20. Write the knowledge-distillation loss and derive the T^2 factor.

$L = (1-a) \cdot \text{CE}(y, p_s) + a \cdot T^2 \cdot \text{KL}(q_T || p_T)$, where $q_T = \text{softmax}(z_t/T)$, $p_T = \text{softmax}(z_s/T)$. Gradient of KL wrt student logits: $(p_T - q_T)/T$ — the $1/T$ shrinks gradients as T grows (softer targets = flatter loss surface). Multiplying the soft term by T^2 keeps its gradient magnitude comparable to the hard term's as you tune T (one T cancels the $1/T$, the other restores the scale that softening removed) — otherwise raising T silently down-weights distillation. Listing 8's student uses exactly $d = (p_T - q_T) \cdot T$ per example, the T^2 -corrected form. Practical T : 2-4; higher T exposes more dark knowledge (relative probabilities of wrong classes) but adds noise.

Q21. Give the DeBERTa idea in two sentences and why it helped.

Disentangled attention: represent each token with SEPARATE content and position vectors, and compute attention as a sum of content-content, content-position, and position-content terms with RELATIVE positions — rather than adding position into the content embedding once at the bottom (BERT) where the two signals blur. Plus an enhanced mask decoder that injects absolute position only near the output. Why it helps: attention can learn "attend to the token two left of me" and "attend to tokens like me" as independent, composable factors — cleaner inductive structure for syntax — and relative positions generalize across sequence lengths (Chapter 16's RoPE/ALiBi argument in encoder form). DeBERTa-v3 (with ELECTRA-style pretraining) remains the strongest encoder recipe.

Q22. What is ELECTRA's objective and why is it more sample-efficient than MLM?

Replaced-token detection: a small generator fills masked positions with plausible tokens; the main model (discriminator) classifies EVERY token as original vs replaced. Supervision density: loss on 100% of positions (vs MLM's 15%) — Listing 6's density argument, engineered into an encoder objective — and the task difficulty auto-curricula with the generator's quality. Result: BERT-class quality at a fraction of the pretraining compute, especially strong at small scale. Connection worth making: it's a GAN-shaped setup WITHOUT adversarial gradients (the generator trains on MLM, not to fool the discriminator — avoiding Chapter 18's instabilities), and DeBERTa-v3 adopted it as the pretraining engine.

Q23. When would you still choose an encoder-decoder over a decoder-only model today?

(1) Long-input, short-output tasks at tight budgets: summarization/translation where the encoder reads the source ONCE bidirectionally and cross-attention gives the decoder full access — no need to carry the source through causal self-attention at every generation step. (2) Structured input-output with strong grounding requirements (data-to-text, semantic parsing) — T5/BART-class models fine-tune extremely well at small scale. (3) Multilingual translation (NLLB-class). (4) When you can fine-tune but not scale: a 770M T5 fine-tuned on the task often beats a far larger decoder prompted zero-shot at fixed serving cost. Decoder-only wins when tasks are open-ended, few-shot, conversational, or you want ONE model for everything. The honest note: at frontier scale the distinction blurred — decoder-only with long context absorbed most encoder-decoder territory.

Q24. How do you evaluate sentence embeddings properly? Name the traps this chapter hit.

Traps from Listing 7: (1) split by DOCUMENT, not chunk — chunk-level splits leak positives across train/test; (2) report chance (0.011 here) — retrieval accuracies are meaningless without it ($0.085 = 8\times$ chance reads very differently from '8.5%'); (3) anisotropy — compare after centering or with recall@k, not raw cosine magnitudes (Chapter 19). Proper protocol: multiple tasks (STS correlation, retrieval recall@k/MRR on BEIR-style suites, clustering purity), against baselines (TF-IDF/BM25 first — often embarrassingly strong; random-encoder control), at matched dimensionality, with the deployment similarity (cosine vs dot changes rankings for unnormalized embeddings). And evaluate on YOUR domain: MTEB rank does not transfer reliably to niche corpora.

Q25. Explain "the transfer set matters more than the labels" for modern LLM practice.

Listing 8's finding scaled up: student quality tracks the SIZE AND COVERAGE of teacher-labeled data more than the form of supervision. Modern instances: small open models trained on frontier-model generations (synthetic instruction data); self-instruct pipelines; constitution-guided self-labeling (Chapter 21); weak-to-strong generalization experiments. Implications: the binding constraints become teacher quality (students inherit teacher errors AND biases — measure, don't assume), transfer-set diversity (coverage of the deployment distribution), and licensing/ToS of teacher outputs. Failure mode to name: distilling on-policy behaviors (refusals, style) without the underlying capability — students that TALK like the teacher but can't reason like it; mitigate with reasoning-trace distillation and verification filtering.

Q26. Why does the induction-head experiment train loss ONLY on the repeated half?

The first half of each sequence is uniform-random — its next-token entropy is maximal and irreducible; loss there is pure noise that (a) swamps the learnable signal in gradient magnitude and (b) rewards degenerate frequency solutions. Masking the loss to the predictable region (Listing 5's `self[:, HALF-1:-1]`) concentrates gradient exactly where the copying rule lives — the model reaches 0.4% loss in 1,200 steps. The verification that nothing leaked: accuracy on the unpredictable half stays at chance ($0.023 \sim 1/40$). General principle for interviews: loss masking is how every structured LM objective works — instruction tuning masks the prompt (Chapter 21), seq2seq masks padding, and getting the mask wrong (training on prompts) is a real and common bug.

Q27. Summarize what reproduces at miniature scale and what demonstrably requires large scale, per this chapter.

Reproduces at 330k params: the transfer advantage (0.563->0.781 at 50 labels — the paradigm itself); neural-over-n-gram perplexity (134 vs 185); induction/copying circuits (99.7% novel-sequence copy with a visible attention stripe); contrastive embedding gains (+45% in 300 steps); distillation via transfer sets (96% of teacher). Requires scale: cloze knowledge (raw predictions are stopwords; topical signal only via frequency adjustment); MLM's bidirectional advantage over CLM (reversed at equal miniature steps); in-context learning of ARBITRARY tasks (only literal copying here); generation coherence beyond a sentence. This partition — mechanisms vs knowledge — is a defensible one-line answer to 'what does scale actually buy?'

Q28. This chapter's measured claims in one pass.

Shared machinery grad-checked $5e-10$ (Listing 1). MLM at 330k params learns the corpus prior first; topical cloze only after frequency adjustment (Listing 2). Transfer: 50 labels 0.563 vs 0.781; pretrained@50 beats scratch@1000 — $\sim 20\times$ label multiplier (Listing 3). CLM ppl 134.3 vs interpolated trigram 184.7, same data and vocab (Listing 4). Induction: 99.7% on novel sequences from a $6.5e12$ space, chance (0.023) on the unpredictable half, $4\times$ -uniform attention stripe (Listing 5). Probing: CLM mean-pool 0.769 > MLM 0.746 > random 0.571 at equal steps — density vs bidirectionality, honestly ordered at small scale; mean-pool > last-token 0.769/0.715 (Listing 6). Sentence embeddings: $8\times \rightarrow 11\times$ chance in 300 contrastive steps, document-held-out (Listing 7). Distillation: soft labels on same 200 examples +0.00; teacher-labeled 3000 unlabeled $\rightarrow 0.632 \rightarrow 0.807 = 96\%$ of teacher (Listing 8).